EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

A cryptocurrency is a digital form of currency that uses blockchain technology to record and verify transactions without requiring a central authority. In this experiment, a simple cryptocurrency and a Proof-of-Work based blockchain are implemented using Python and Flask, with multiple nodes communicating through a peer-to-peer network.

- **Blockchain:** A blockchain is a sequence of blocks containing transactions. Each block is linked to the previous block using its cryptographic hash, making the data difficult to modify.
- **Mining and Proof-of-Work:** Mining is the process of finding a valid proof that satisfies a predefined condition. In this experiment, miners repeatedly calculate SHA-256 hashes until they find a hash beginning with a specific number of zeros.
- **Transactions and Mempool:** Transactions represent the transfer of cryptocurrency between users. Pending transactions are temporarily stored in the mempool and are added to a block when a miner successfully mines a new block.
- **Peer-to-Peer Network and Consensus:** Multiple blockchain nodes communicate with each other to maintain copies of the blockchain. The consensus mechanism allows a node to replace its blockchain with the longest valid chain available from its connected peers.

Colab Link: Blockchain_Exp3.ipynb

Code

1. **Define the Blockchain Class**

```
# Blockchain class
# This class contains the complete blockchain functionality

class Blockchain:

    def __init__(self):

        # List to store all blocks
        self.chain = []

        # List to store pending transactions
        # These transactions will be included in the next block
        self.current_transactions = []

        # Set of connected peer nodes
        self.nodes = set()

        # Create the first block (Genesis Block)
        self.new_block(previous_hash='1', proof=100)
```

2. Setting up the Flask Server

```
from flask import Flask, jsonify, request

def create_node(port):

    app = Flask(__name__)

    # Create blockchain object
    blockchain = Blockchain()

    # Generate a unique ID for this node
    node_identifier = str(uuid.uuid4()).replace('-', '')

    @app.route('/mine', methods=['GET'])

    def mine():

        # Get proof of work
        last_block = blockchain.last_block
        last_proof = last_block['proof']

        proof = blockchain.proof_of_work(last_proof)

        # Mining reward
        blockchain.new_transaction(
            sender="0",
            recipient=node_identifier,
            amount=1
        )
```

3. Mining Logic

```
# Get the proof of work for the previous block

last_block = test_blockchain.last_block

last_proof = last_block['proof']

print("Mining started...")

proof = test_blockchain.proof_of_work(last_proof)

print("Mining completed!")
print("Proof:", proof)
```

```
Mining started...
Mining completed!
Proof: 35293
```

4. Setup Multiple Web Nodes

```
# Create Flask application for Node 5001

app5001 = create_node(5001)
```

```
# Create Flask application for Node 5002

app5002 = create_node(5002)
```

```
# Create Flask application for Node 5003

app5003 = create_node(5003)
```

5. Verify Node Status

```
 * Serving Flask app '__main__'
 * Debug mode: off
 * Serving Flask app '__main__'
 * Debug mode: off
 * Serving Flask app '__main__'
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://172.28.0.12:5001
 * Debug mode: off
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://172.28.0.12:5002
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://172.28.0.12:5003
INFO:werkzeug:Press CTRL+C to quit
Node 5001 started
Node 5002 started
Node 5003 started
```

6. Connect the Nodes

```
# Node 5001 connects to Node 5002 and Node 5003

response = requests.post(
    "http://127.0.0.1:5001/nodes/register",
    json={
        "nodes": [
            "http://127.0.0.1:5002",
            "http://127.0.0.1:5003"
        ]
    }
)

print(response.json())
```

```
{'message': 'New nodes have been added', 'total_nodes': ['127.0.0.1:5002', '127.0.0.1:5003']}
```

7. Add Transactions to the Network

```
# Add a transaction to Node 5001

transaction = {
    "sender": "Aarya",
    "recipient": "Sanket",
    "amount": 20
}

response = requests.post(
    "http://127.0.0.1:5001/transactions/new",
    json=transaction
)

print(response.json())
```

```
{'message': 'Transaction will be added to Block 4'}
```

8. Mine a Network Block

```python
# Mine the pending transactions on Node 5001
print("Mining block...")

response = requests.get(
    "http://127.0.0.1:5001/mine"
)
result = response.json()

print(
    json.dumps(
        result,
        indent=4
    )
)
```

```
Mining block...
{
    "block": {
        "index": 4,
        "previous_hash": "2f17f222cb41cc8771b560a328bf1489cd9e41b35f4b1a22151ec9d8f48af5ca",
        "proof": 119678,
        "timestamp": 1787882289.4174988,
        "transactions": [
            {
                "amount": 500,
                "recipient": "Atharva",
                "sender": "Varun"
            },
            {
                "amount": 20,
                "recipient": "Sanket",
                "sender": "Aarya"
            },
            {
                "amount": 1,
                "recipient": "Chinmay",
                "sender": "Nick"
            }
        ]
    },
    "message": "New Block Mined"
}
```

```python
# Fetch the complete blockchain from Node 5001
response = requests.get(
    "http://127.0.0.1:5001/chain"
)

chain_data = response.json()

print(
    json.dumps(
        chain_data,
        indent=4
    )
)
```

```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "1",
            "proof": 100,
            "timestamp": 1787801701.6819193,
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499",
            "proof": 35293,
            "timestamp": 1787802260.2626357,
            "transactions": [
                {
                    "amount": 50,
                    "recipient": "Chinmay",
                    "sender": "Aarya"
                },
                {
                    "amount": 500,
                    "recipient": "Sanket",
                    "sender": "Atharva"
                },
                {
                    "amount": 20,
                    "recipient": "Nick",
                    "sender": "Sunchu"
                },
                {
                    "amount": 1,            1704cd690a19c019f4f5925",
                    "recipient": "c09b54da7
            ]
        }
}
```


9. Synchronize the Network (Consensus)


```
{
   "message": "Our chain was replaced",
   "new_chain": [
        {
            "index": 1,
            "previous_hash": "1",
            "proof": 300,
            "timestamp": 1787801701.6819193,
            "transactions": []
        },
        {
            "index": 2,
            "previous_hash": "ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499",
            "proof": 35293,
            "timestamp": 1787802260.2626357,
            "transactions": [
                {
                    "amount": 50,
                    "recipient": "Chinmay",
                    "sender": "Aarya"
                },
                {
                    "amount": 500,
                    "recipient": "Sanket",
                    "sender": "Atharva"
                },
                {
                    "amount": 20,
                    "recipient": "Nick",
                    "sender": "Sunchu"
                },
                {   "amount": 1,
                    "recipient": "cc9b54da71704cd690a19c019f4f5925",
                    "sender": "0"
            ]
        },
        {   "index": 3,
            "previous_hash": "fbe2617ff02aadba2d3edaee4f1c2e9f5f6d7d998da748d5b5405510@cfc2",
            "proof": 35689,
            "timestamp": 1787802278.0779805,
            "transactions": [
                {   "amount": 1,
                    "recipient": "cc9b54da71704cd690a19c019f4f5925",
                ]   "sender": "0"
        },
        {   "index": 4,
            "previous_hash": "2f17f222cb41cc8771b560aa20bf1489cd9e41b35fab1a22151ec9d8f48af5ca",
            "proof": 119678,
            "timestamp": 1787802289.4174988,
            "transactions": [
                {   "amount": 500,
                    "recipient": "Atharva",
                    "sender": "Varun"
                },  "amount": 20,
                {   "recipient": "Sanket",   { meodent": 1,    cc9b54da71704cd690a19c019f4f5925",
                    "sender": "Aarya"
}
```

```
{
  "message": "Our chain was replaced",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "1",
      "proof": 100,
      "timestamp": 1787801701.6819193,
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499",
      "proof": 35293,
      "timestamp": 1787802260.2626357,
      "transactions": [
        {
          "amount": 50,
          "recipient": "Chinmay",
          "sender": "Aarya"
        },
        {
          "amount": 500,
          "recipient": "Sanket",
          "sender": "Atharva"
        },
        {
          "amount": 20,
          "recipient": "Nick",
          "sender": "Sunchu"
        },
        {
          "amount": 1,
          "recipient": "cc9b54da71704cd690a19c019f4f5925",
          "sender": "0"
        }
      ]
    },
    {
      "index": 3,
      "previous_hash": "fbe2617ff02aadba2d3edaee4f1c2e9f5f6d7d998da748d5b5405510bcfc2",
      "proof": 35689,
      "timestamp": 1787802278.0779805,
      "transactions": [
        {
          "amount": 1,
          "recipient": "cc9b54da71704cd690a19c019f4f5925",
          "sender": "0"
        }
      ]
    },
    {
      "index": 4,
      "previous_hash": "2f17f222cb41cc8771b560aa20bf1489cd9e41b35fab1a22151ec9d8f48af5ca",
      "proof": 119678,
      "timestamp": 1787802289.4174988,
      "transactions": [
        {
          "amount": 500,
          "recipient": "Atharva",
          "sender": "Varun"
        },
        {
          "amount": 20,
          "recipient": "Sanket",
          "sender": "Aarya"
        }
      ]
    }
  ]
}
```


10. Fetch chains from all three nodes

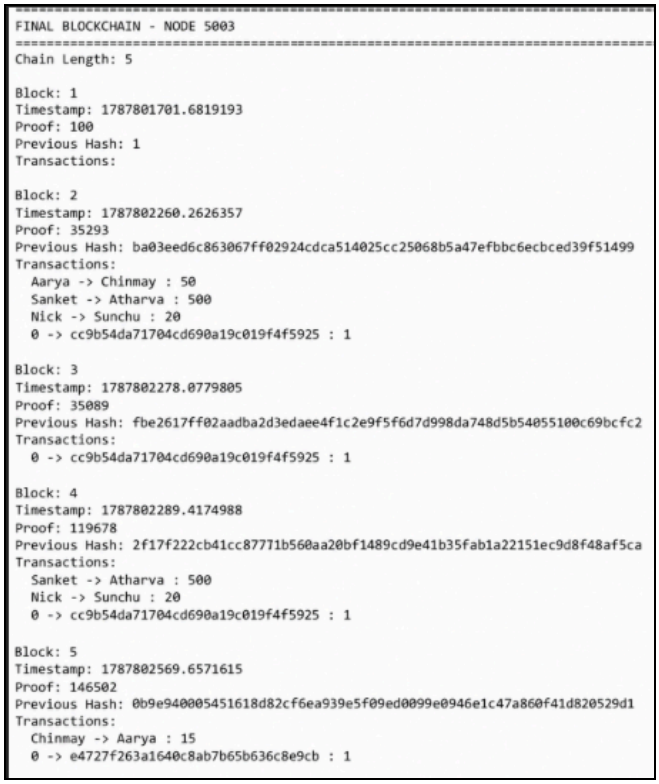
```python
# Fetch chains from all three nodes

for port in [5001, 5002, 5003]:

    response = requests.get(
        f"http://127.0.0.1:{port}/chain"
    )

    data = response.json()

    print("=" * 60)
    print(f"NODE {port}")
    print("Chain Length:", data['length'])

    for block in data['chain']:

        print(
            f"Block {block['index']} | "
            f"Transactions: {len(block['transactions'])} | "
            f"Proof: {block['proof']}"
        )
```

```
============================================================
NODE 5001
Chain Length: 4
Block 1 | Transactions: 0 | Proof: 100
Block 2 | Transactions: 4 | Proof: 35293
Block 3 | Transactions: 1 | Proof: 35089
Block 4 | Transactions: 3 | Proof: 119678
============================================================
NODE 5002
Chain Length: 4
Block 1 | Transactions: 0 | Proof: 100
Block 2 | Transactions: 4 | Proof: 35293
Block 3 | Transactions: 1 | Proof: 35089
Block 4 | Transactions: 3 | Proof: 119678
============================================================
NODE 5003
Chain Length: 4
Block 1 | Transactions: 0 | Proof: 100
Block 2 | Transactions: 4 | Proof: 35293
Block 3 | Transactions: 1 | Proof: 35089
Block 4 | Transactions: 3 | Proof: 119678
```

11. Final blockchain of all three nodes

```
FINAL BLOCKCHAIN - NODE 5001
================================================================
Chain Length: 5

Block: 1
Timestamp: 1787801701.6819193
Proof: 100
Previous Hash: 1
Transactions:

Block: 2
Timestamp: 1787802260.2626357
Proof: 35293
Previous Hash: ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499
Transactions:
  Aarya -> Chinmay : 50
  Sanket -> Atharva : 500
  Sunchu -> Nick : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 3
Timestamp: 1787802278.0779805
Proof: 35089
Previous Hash: fbe2617ff02aadba2d3edaee4f1c2e9f5f6d7d998da748d5b54055100c69bcfc2
Transactions:
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 4
Timestamp: 1787802289.4174988
Proof: 119678
Previous Hash: 2f17f222cb41cc8771b560aa20bf1489cd9e41b35fab1a22151ec9d8f48af5ca
Transactions:
  Sanket -> Atharva : 500
  Sunchu -> Nick : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 5
Timestamp: 1787802569.6571615
Proof: 146502
Previous Hash: 0b9e940005451618d82cf6ea939e5f09ed0099e0946e1c47a86f41d820529d1
Transactions:
  Sunchu -> Chinmay : 15
  0 -> e4727f263a1640c8ab7b65b636c8e9cb : 1
```

```
================================================================
FINAL BLOCKCHAIN - NODE 5002
================================================================
Chain Length: 5

Block: 1
Timestamp: 1787801701.6819193
Proof: 100
Previous Hash: 1
Transactions:

Block: 2
Timestamp: 1787802260.2626357
Proof: 35293
Previous Hash: ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499
Transactions:
  Aarya -> Chinmay : 50
  Sanket -> Atharva : 500
  Nick -> Sunchu : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 3
Timestamp: 1787802278.0779805
Proof: 35089
Previous Hash: fbe2617ff02aadba2d3edaee4f1c2e9f5f6d7d998da748d5b54055100c69bcfc2
Transactions:
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 4
Timestamp: 1787802289.4174988
Proof: 119678
Previous Hash: 2f17f222cb41cc8771b560aa20bf1489cd9e41b35fab1a22151ec9d8f48af5ca
Transactions:
  Sanket -> Atharva : 500
  Nick -> Sunchu : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 5
Timestamp: 1787802569.6571615
Proof: 146502
Previous Hash: 0b9e940005451618d82cf6ea939e5f09ed0099e0946e1c47a86f41d820529d1
Transactions:
  Chinmay -> Aarya : 15
  0 -> e4727f263a1640c8ab7b65b636c8e9cb : 1
```



```
================================================================================
FINAL BLOCKCHAIN - NODE 5003
================================================================================
Chain Length: 5

Block: 1
Timestamp: 1787801701.6819193
Proof: 100
Previous Hash: 1
Transactions:

Block: 2
Timestamp: 1787802260.2626357
Proof: 35293
Previous Hash: ba03eed6c863067ff02924cdca514025cc25068b5a47efbbc6ecbced39f51499
Transactions:
  Aarya -> Chinmay : 50
  Sanket -> Atharva : 500
  Nick -> Sunchu : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 3
Timestamp: 1787802278.0779805
Proof: 35089
Previous Hash: fbe2617ff02aadba2d3edaee4f1c2e9f5f6d7d998da748d5b54055100c69bcfc2
Transactions:
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 4
Timestamp: 1787802289.4174988
Proof: 119678
Previous Hash: 2f17f222cb41cc87771b560aa20bf1489cd9e41b35fab1a22151ec9d8f48af5ca
Transactions:
  Sanket -> Atharva : 500
  Nick -> Sunchu : 20
  0 -> cc9b54da71704cd690a19c019f4f5925 : 1

Block: 5
Timestamp: 1787802569.6571615
Proof: 146502
Previous Hash: 0b9e940005451618d82cf6ea939e5f09ed0099e0946e1c47a860f41d820529d1
Transactions:
  Chinmay -> Aarya : 15
  0 -> e4727f263a1640c8ab7b65b636c8e9cb : 1
```

Conclusion

In this experiment, a simple cryptocurrency and blockchain network were successfully implemented using Python, Flask, and Proof-of-Work. Multiple nodes were connected in a peer-to-peer network to perform transactions, mine blocks, and maintain the blockchain. The experiment demonstrated how transactions are stored in the mempool, added to blocks through mining, and synchronized across nodes using the longest valid chain consensus mechanism.